Language Model

Yuchen Hou

June 9, 2024

Abstract

The basic script employs the Wikitext-2 dataset to train multi-layer RNN, GRU, LSTM, and Transformer language models for text generation. I have tuned the provided code framework, visualized the changes in loss and perplexity (PPL) during the training process, discussed optimization methods for the Transformer model, and incorporated the GPT language model.

1 Introduction

1.1 PPL

I use *perplexity* (PPL) to measure the model performance, which shows the average divergence of each prediction. Suppose that we have an N-word sentence $S = \{w_1, \ldots, w_N\}$, then

$$\begin{aligned} PPL(S) &= (P(w_1 \ldots w_N))^{-1/N} \\ &= \left(\prod_{i=1}^{N} P(w_i | w_1 \ldots w_{i-1}) \right)^{-1/N} \\ &= \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \log P(w_i | w_1 \ldots w_{i-1}) \right) \end{aligned}$$

1.2 RNNs

Recurrent Neural Networks (RNNs) are a class of neural networks designed for sequential data processing. They are characterized by their ability to maintain a hidden state from one time step to another, allowing them to capture temporal dependencies in data. However, RNNs often struggle with long-term dependencies due to vanishing and exploding gradient problems.

LSTMs are an extension of RNNs that include mechanisms called gates—input, forget, and output gates. These gates regulate the flow of information and are designed to address the vanishing gradient problem, enabling the model to learn longer sequences.

GRUs simplify the LSTM architecture by combining the input and forget gates into a single "update gate" and merging the cell state and hidden state. This reduction in complexity allows GRUs to perform comparably to LSTMs with fewer parameters.

1.3 Transformer

The Transformer model moves away from recurrence entirely and relies solely on attention mechanisms to draw global dependencies between input and output. The Transformer's architecture allows for significantly more parallelization than RNNs and has been found to be very effective, particularly in tasks like machine translation and text generation.

2 Experiment

We utilize the GigaSpeech dataset, which is partitioned into training, validation, and test sets. Initially, we load these datasets and construct a dictionary mapping words to unique integer identifiers. Subsequently, the dataset is segmented into batches of size = 20 in our baseline training setup.

Each batch of data is fed into the model to produce outputs, and the discrepancy between predicted outputs and actual data is quantified using the Negative Log Likelihood Loss. The optimization process involves gradient clipping at a threshold of 0.25, which means any gradient less than 0.25 is adjusted to 0.25. The adjusted gradients are then scaled by the learning rate, initially set to 20 and reduced to a quarter of its previous value if no improvement is observed in the validation set performance.

2.1 Baseline

The training is conducted over 20 epochs, with the size of the model parameters maintained below 60M. The baseline settings is shown in Tbale 1

Parameters	value
model	LSTM
embedding size	200
hidden unit num	200
layer num	2
learning rate	20
gradient clipping	0.25
sequence length	35
dropout	0.2

Table 1: Baseline settings

2.2 Hyper parameters optimization

2.2.1 Model parameters

1. **Model type:**As shown in Table 2 and Figure 1, the LSTM model exhibits the best performance with the lowest PPL and Loss, suggesting its high efficiency in sequence prediction. The GRU model follows closely, though with a higher PPL and Loss, indicating a slight reduction in prediction accuracy and efficiency.

Model	PPL	Loss
LSTM	5.04	155.23
GRU	5.78	325.18
Transformer	6.62	753.49
RNN_TAHN	8.87	7100.68
RNN_RELU	n/a	n/a

Table 2: Model type

2. **Layer number:** As I chage the number of layers on the LSTM model, the results in shown in the table3. 3 layers is the best.

layer num	ppl	loss
1	5.10	163.99
2 *	5.04	155.23
3	5.01	156.46
5	5.08	160.87

Table 3: Caption

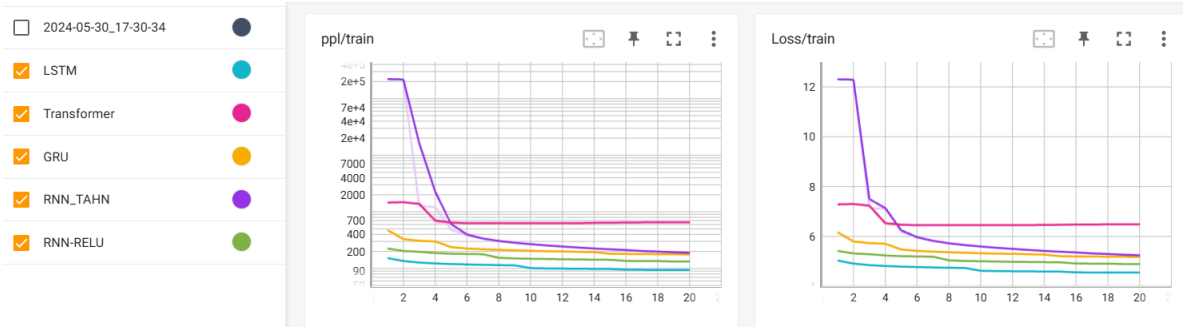

Figure 1: Model type

3. **Hidden unit:** As I change the number of hidden units in the LSTM model, the results in shown in the table4.As the number of hidden units gets larger, the performance gets better.

layer num	ppl	loss
100	5.19	178.78
200 *	5.04	155.23
400	4.94	139.15
650	**4.87**	**130.89**

Table 4: Caption

4. **n head:** As I change the number of head in Transformer model, the performance didn't better, which is kind of weird. The results is shown in Table 5

num head	ppl	loss
2	6.58	720.20
5	6.62	753.49
10	6.61	742.28

Table 5: Caption

5. **Embedding size and sequence length:** Model performance gets better as the enbedding size anf sequence length get larger. The details are shown in Table 6

2.2.2 Hyper parameters

As the details are shown in Table 7, I optimize the LSTM model by adjusting some hyper parameters.

- As I decrease the dropout from 0.2 to 0.1, both the ppl and test loss get lower.
- Add tied can better the performance.
- Adjusting learning rate has the potential to improve model performance regardless of increase or decrease.
- Smaller clips reduce ppl and test loss.

2.3 About Transformer

During recent experiments, an unexpected outcome was observed where RNN-based models outperformed Transformer-based models. This can be attributed to several factors. Primarily, the datasets used consisted of short spoken sentences with incorrect tags and obscure vocabulary, hindering the Transformer models in effectively capturing context. Moreover, Transformer models in these tests had fewer parameters, limiting their scalability and contextual understanding compared to RNNs.

result	emb = 200 *	emb=400	emb = 650	bptt=50	bptt=100
ppl	5.04	5.04	5.04	5.01	5.00
loss	155.23	154.71	154.42	149.40	147.80

Table 6: Caption

Configuration	PPL	Loss
Baseline (dropout=0.2, lr=20, clip=0.25)	5.04	155.23
Dropout = 0.1	5.03	152.54
Tied	4.97	143.40
Clip 0.1	4.99	146.42
Clip 0.5	5.09	162.43
Learning Rate = 10	5.00	147.83
Learning Rate = 30	4.99	147.27

Table 7: Hyperparameter optimization results for the LSTM model

2.4 GPT model

The Generative Pre-trained Transformer (GPT) and the traditional Transformer models both leverage the power of attention mechanisms, yet they are architecturally distinct in their applications and focus.

GPT exclusively utilizes the Transformer's decoder architecture. Unlike the full Transformer, which includes both encoder and decoder components for tasks like translation, GPT uses a decoder-only structure optimized for generative tasks.

GPT integrates token embeddings and positional encodings similarly to the Transformer. However, GPT's application of these components is geared towards text generation:

$$\text{emb}(x_i) = E_{x_i} + P_i \tag{1}$$

Where E_{x_i} represents token embeddings and P_i is the positional encoding for position i.

While both models use self-attention, GPT's decoder layers are designed for autoregressive tasks, generating one token at a time:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2}$$

This is in contrast to the Transformer, which processes entire sequences simultaneously in its encoder-decoder structure.

Due to time constraints I didn't use the pre-trained GPT model, making my training results on GPT similar to Transformer. The details are shown in the Figure 2.4

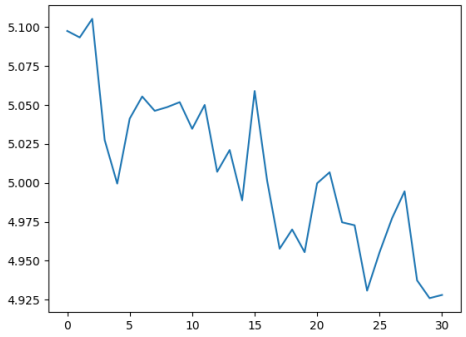

Figure 2: GPT loss

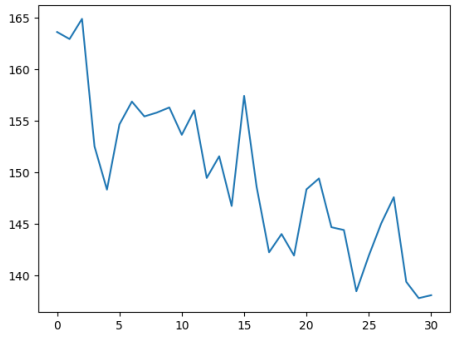

Figure 3: GPT ppl